Workstation Factory

Operator manual

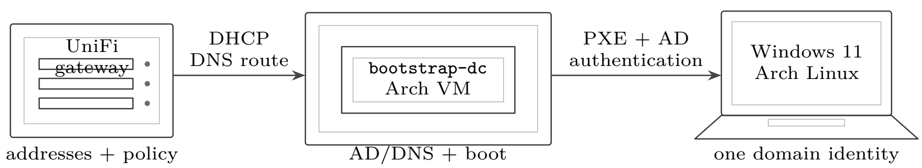
From an isolated bootstrap VM to repeatable Windows 11 and Arch laptops

This manual creates a dual-boot laptop that can authenticate the same person under Windows 11 Pro or Arch Linux. It starts with an isolated Arch virtual machine named `bootstrap-dc`; it does not require a permanent Controller. The public procedure uses placeholders. Put household names, addresses, machine inventory and encrypted secret material in the private instance repository.

PHASE-1 BOUNDARY

UniFi remains the only DHCP authority. The Controller hosts Samba AD with integrated DNS, and HTTP/iPXE services. Workstation disks are deliberately **not encrypted**; Secure Boot is deliberately disabled. Cached domain login may work indefinitely while a laptop is away, so a disconnected laptop cannot be remotely revoked. Treat every phase-1 workstation as unsuitable for unrecoverable or sensitive data.

The result



Stable names and AD discovery decouple each workstation from the temporary VM.

Identity	The permanent DNS domain comes from `identity_dns_domain`. Clients discover domain controllers through DNS service records, never a baked-in machine address.
Boot	Windows is the default after a five-second UEFI boot menu; Arch remains independently bootable.
Storage	The default ratio is 75 percent Windows and 25 percent Arch after fixed partitions, subject to 160 GiB and 64 GiB minimums.
Network home	A local profile always opens. Optional SMB storage mounts afterward and fails soft.

Stage 0 — Record inputs without secrets

Create a private instance record before changing the network. It names the site prefix, VLANs, subnets, gateway, Controller addresses, workstation hostname, disk serial, desired layout and physical switch ports. It contains no plaintext password, Wi-Fi key, recovery material or reusable join token.

Public constant	Private instance value	Secret channel
`identity_dns_domain`; Windows 11 Pro; UEFI	Addresses, hostnames, serials, MACs, port assignments	AD bootstrap password, join credential, Wi-Fi key
75/25 default; 160/64 GiB floors	Approved per-machine override and disk identity	Local rescue and user passwords
`boot_service_fqdn` alias	Current server behind the alias	NAS and deployment credentials

Answer before continuing: Does the machine name satisfy both DNS and Windows naming rules, and is it absent from AD and DNS? Does the disk's physical label match the recorded model, serial and capacity? Who owns the switch port and DHCP scope?

Stage 1 — Build the isolated bootstrap VM

Run the repository dependency target on an Arch build host, then create the virtual machine. The VM has four vCPUs, 8 GiB RAM and an 80 GiB disk. During this stage it attaches only to an isolated virtual network.

```
make homelab-bootstrap-deps
make homelab-bootstrap-vm-create APPLY=1
make homelab-bootstrap-vm-run APPLY=1
make homelab-bootstrap-controller INVENTORY=<private-inventory>
```

Expect: The VM reports hostname `bootstrap_dc_fqdn`; Samba, Kerberos and DNS tests pass; no packet from the VM reaches the household LAN.

If not: If a target is missing, reports a proposed command instead of running it, or cannot prove isolation, stop. Do not improvise package commands or bridge the VM manually; fix the reproducible target.

The first proof is negative: the unfinished services cannot reach the live LAN.

`make homelab-test`	The repository's VM, identity, PXE and workstation tests pass; save the test report and artifact digests.
`make homelab-bootstrap-vm-status`	Reports the expected isolated VM state without changing the VM or UniFi.

Stage 2 — Establish the permanent identity plane

The first domain controller creates the permanent forest `identity_dns_domain`, `kerberos_realm`, and `netbios_name`. The Controller's implementation may later move to a VM or physical machine; those names do not change.

DNS IS PART OF ACTIVE DIRECTORY

Windows and Arch locate LDAP and Kerberos through DNS SRV records. All domain-member workstation interfaces must query AD DNS. The AD DNS service may forward unrelated queries, but a public resolver cannot replace it.

`samba-tool domain level show`	The forest and domain levels are reported without error.
`host -t SRV _ldap._tcp.⟨identity-domain⟩`	At least one domain controller target and port 389 are returned.
`host -t SRV _kerberos._udp.⟨domain⟩`	At least one domain controller target and port 88 are returned.
`kinit <domain-admin> && klist`	A ticket for `⟨kerberos-realm⟩` exists; destroy it after the test with `kdestroy`.

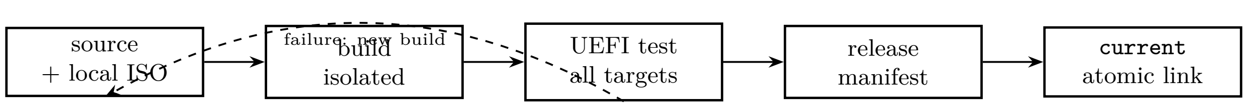
Stage 3 — Build versioned boot targets

Build each payload independently, then build the signed-off set. The Windows target consumes a locally supplied Microsoft ISO and never redistributes it. Every release directory is immutable and contains a SHA-256 manifest.

```
make homelab-pxe-controller
make homelab-pxe-arch
make homelab-pxe-windows WINDOWS_ISO=/private/path/windows-11.iso
make homelab-pxe-all
make homelab-pxe-test
```

Expect: The test boots each menu entry in an isolated UEFI virtual machine, fetches every payload over HTTP, verifies each digest, and stops before disk writes unless a disposable test disk and full serial authorization are present.

If not: A missing Microsoft driver, changed ISO digest or failed fetch invalidates the release. Never edit an artifact in place; repair the source and build a new version.



Stage 4 — Choose the real network attachment

This is a deployment gate, not a build prerequisite. Choose exactly one: existing LAN, a dedicated second interface, or a VLAN trunk. Record the reason, the recovery path and the observed switch-port configuration. UniFi remains the sole address authority in every choice.

RECOMMENDED PRODUCTION BOUNDARY

Use a dedicated provisioning VLAN with UniFi DHCP options pointing UEFI clients at the stable boot endpoint. Permit only AD/DNS, boot HTTP, time, approved package sources and explicitly tested storage. A final deny rule is added only after each required flow passes independently.

Attachment	What it buys	Cost or risk
Existing LAN	Fastest first hardware trial	Weakest destructive-install boundary; noisy policy
Dedicated interface	Existing host path stays untouched	Needs a second cable and switch port
VLAN trunk	Several isolated services over one cable	Bridge and recovery procedure are more complex

`make homelab-workstation-verify INSTANCE=<private-acceptance.json>` — Validates the recorded DHCP, AD DNS, boot HTTP, time and cross-zone acceptance evidence without changing the network.

Answer before continuing: Which physical port is the build port? Can an ordinary household client see the installer? Can a build client reach a management interface? Is there a tested local-console recovery path if the bridge is wrong?

Stage 5 — Configure UniFi, one dependency at a time

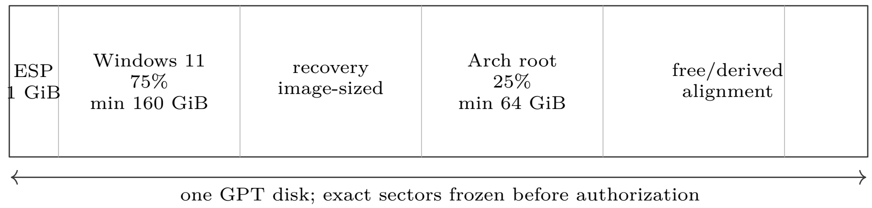
Create the small provisioning network at the expected device count, leaving adjacent space available. Prefer simple human-facing gateway and service addresses such as `.1`; the actual CIDR boundary remains technically valid. Configure DHCP, AD DNS, time and internet update access before setting network boot options. Add isolation rules last.

1. Create the VLAN and subnet. *This gives build traffic a bounded address space.* Verify one ordinary lease, gateway reachability and the exact CIDR.
2. Set the network's DNS server to the AD DNS address. *This enables domain service discovery.* Verify the LDAP and Kerberos SRV records from a client on that VLAN.
3. Permit time synchronization. *Kerberos rejects sufficiently skewed clocks.* Verify both client and DC report synchronized time.
4. Configure network boot for x86-64 UEFI, with the Controller address and iPXE filename. *This hands firmware to the versioned HTTP boot chain.* Verify a test VM or laptop reaches the menu.
5. Permit only required update and service flows. *This makes installation useful without exposing management networks.* Test each allow rule before the final deny.

If not: If isolation breaks a client, inspect DHCP, DNS, time, then the specific destination port. Do not disable the entire policy as a diagnostic shortcut; enable or log one dependency at a time.

Stage 6 — Mint one disposable pilot

Connect the laptop through a known UEFI-PXE-capable USB-C Ethernet adapter. Disable Secure Boot, select UEFI network boot, and choose the approved workstation profile. The installer asks for the bespoke hostname, disk serial, Windows/Arch allocation and default OS. It rejects a disk that cannot satisfy both filesystem minimums.



The ratio is a policy default, never permission to violate either minimum.

DESTRUCTIVE AUTHORIZATION

The operator must compare the visible physical device with model, serial and capacity, then type the complete disk serial. A profile selection, hostname or `yes` is not authorization.

Expect: Windows Setup completes first; Arch fills its declared partition; both write independent UEFI entries; Windows becomes the default after a five-second menu. The system does not join the domain until both native boots pass.

`efibootmgr -v`	Windows and Arch entries both exist; the documented default points to the five-second selector.
`lsblk -o NAME,SIZE,FSTYPE,PARTLABEL,UUID`	Observed partitions match the frozen preflight plan within declared alignment tolerances.

Pilot installation — measure, write, prove

The profile is a proposed geometry, not permission to erase a disk. Print the plan, then compare it with the laptop in front of you. Use sectors as the authoritative unit; rounded GiB figures are only a human cross-check.

1. Photograph the firmware summary and the physical disk label if it is accessible. Record firmware revision, disk model, serial and advertised capacity.
2. Boot the approved PXE release in UEFI mode. Record the release version and manifest digest shown by the installer.
3. Run the read-only preflight. Save `lsblk -b -o NAME,SIZE,MODEL,SERIAL,TYPE,TRAN` and `wipefs -n` output.
4. Compare the discovered logical-sector size, last usable sector and disk serial with the frozen plan. Differences are a stop condition.

Answer before continuing: Is this the one authorized internal disk? Is the PXE or USB medium absent from the target list? Does the disk have at least the Windows and Arch minimums after the ESP, Microsoft reserved partition and recovery allowance? Are there files on the existing disk that have not been independently recovered and opened?

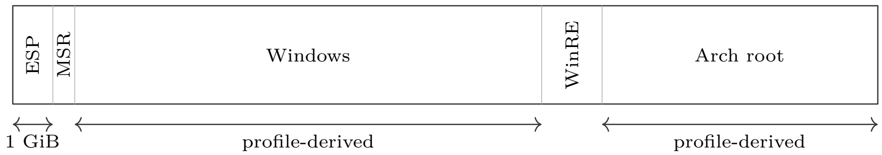
Measurement	Frozen value	Observed value	Pass rule
Disk model and serial	⟨`plan`⟩	⟨`preflight`⟩	Exact serial; model consistent
Disk bytes	⟨`plan`⟩	⟨`lsblk`⟩	Exact byte count
Logical sector bytes	⟨`plan`⟩	⟨`blockdev`⟩	Exact match
First/last usable LBA	⟨`plan`⟩	⟨`sgdisk`⟩	Exact match
PXE release digest	⟨`release`⟩	⟨`screen`⟩	Exact SHA-256

LAST REVERSIBLE POINT

Do not type the disk serial until the table passes. The next operation destroys the selected partition table. No hostname, device name such as `/dev/nvme0n1`, or general confirmation substitutes for the complete serial.

Create and verify the GPT

The automation creates, in order, one shared EFI System Partition (ESP), one Microsoft Reserved partition (MSR), the Windows partition, Windows recovery, and the Arch root partition. Start each resizable partition on the alignment boundary emitted by the profile. Do not calculate boundaries by eye in a GUI.



One GPT and one shared ESP; exact LBA boundaries come from the signed plan.

Expect: A second read of the GPT reports the planned partition count, type GUIDs, start sectors and end sectors. No filesystem signature exists outside the partitions declared by the profile.

If not: If any start sector, type GUID, serial or disk byte count differs, stop before formatting. Save both observations, remove installer power, and regenerate the plan for the hardware actually present.

Install Windows first

Boot Windows Setup from the versioned PXE entry. Load only the driver bundle whose hardware identifiers and digest were approved with the release. At the disk chooser, select the pre-created partition labelled for Windows; do not delete the table and do not allow Setup to consume the Arch partition.

1. Confirm Setup reports the planned Windows partition size within its display-rounding tolerance. Ask: “Which partition will Setup format, and what two observed facts identify it?”
2. Install Windows 11 Pro. Firmware activation may occur later; activation status is not an installation gate.
3. At the first desktop, install firmware and drivers from the laptop vendor or Windows Update. Record each device that retains a warning icon.
4. Apply all update rounds, restarting and checking again until two successive scans report no required update. Optional preview drivers remain excluded unless a recorded hardware fault requires one.

`Get-Disk | Format-Table Number,FriendlyName,SerialNumber,Size`	The system disk serial and byte count identify the authorized disk.
`Get-Partition | Sort-Object Offset`	Partition offsets and sizes match the frozen plan; no unexpected data partition exists.
`Get-PnpDevice | Where-Object Status -ne 'OK'`	Returns no unexplained device; record any intentional disabled device.
`Confirm-SecureBootUEFI`	Reports `False`, consistent with the phase-1 boundary; an unsupported result is investigated, not assumed.
`manage-bde -status`	All workstation volumes report protection off; device encryption must not have silently enabled itself.

WINDOWS RECOVERY PROOF

Open the Windows Recovery Environment once before installing Arch. Confirm the keyboard works and that Startup Repair and Command Prompt are visible. Exit without changing the disk. This proves the recovery partition is bootable before another operating system changes UEFI entries.

Install Arch into its declared partition

Return to the same PXE release and select Arch. Re-run disk identity preflight; a successful Windows boot does not waive serial authorization. Format only the Arch root partition. Mount the existing ESP at `/efi`; never format it.

1. Check time synchronization before package installation. Record clock offset and time source; Kerberos will depend on this later.
2. Mount Arch root, then mount the existing FAT32 ESP. List `/efi/EFI/Microsoft/Boot/bootmgfw.efi`; its presence is a stop-check before installing a boot manager.
3. Install the base system, Intel microcode, NetworkManager, an LTS-capable kernel path, and the approved graphics, Wi-Fi and firmware packages.
4. Generate `/etc/fstab` by UUID. Inspect every line: root must use the Arch filesystem UUID and the ESP must use the shared FAT UUID.

5. Install `systemd-boot` without removing the Microsoft directory. Create the Arch entry, set a five-second timeout, and set Windows `auto-windows` as the default.

`findmnt -verify -verbose`	Reports no fstab parse or unreachable source error.
`bootctl status`	Shows the shared ESP, an Arch entry and the automatic Windows entry; default is Windows and timeout is five seconds.
`Verify the Windows EFI loader exists`	Confirm `/efi/EFI/Microsoft/Boot/bootmgfw.efi` remains present after the Arch boot manager is installed.
`lspci -k`	Each storage, graphics, Ethernet and wireless controller has the expected kernel driver bound.
`systemctl -failed`	Returns no unexplained failed unit after a native Arch boot.

If not: If the automatic Windows entry is absent, do not hand-edit or copy Microsoft binaries. Verify the ESP mount and the original Microsoft path. If Windows files are missing, use Windows recovery media to repair them, then rerun `bootctl status`.

Prove both native boot paths

Power the laptop fully off between tests. A reboot alone may preserve firmware or device state and conceal a cold-start defect.

1. Power on without touching a key. Measure from menu appearance until Windows begins loading. Pass only if Windows is selected after five seconds.
2. Restart, select Arch, and measure time to a local login prompt. Record network association time separately from operating-system boot time.
3. Use the firmware's one-time boot menu to start Windows Boot Manager directly, then to start Linux Boot Manager directly.
4. Repeat once with Ethernet disconnected. Both local operating systems must reach login independently of PXE, AD, DNS and network storage.

Boot path	**Expected**	**Measured**	**Evidence**
Default menu	Windows, 5 s		video / stopwatch
Selected Arch	local login		journal boot ID
Firmware Windows entry	Windows login		firmware photo
Firmware Linux entry	Arch login		firmware photo
Both, cable absent	local login		two boot IDs

Recovery order

Recovery preserves the other operating system before it improves convenience. Do not repartition to repair a boot menu.

1. **Firmware first:** open the one-time menu. If either native entry works, boot it and record `efibootmgr -v` or `bcdedit /enum firmware`.
2. **Selector second:** from Arch media, mount the existing root and ESP, confirm their UUIDs, chroot, then reinstall systemd-boot and regenerate the Arch entry. Do not format either filesystem.
3. **Windows files third:** if the Microsoft EFI binary is absent or Windows reports boot damage, use the matching Windows 11 recovery image and `bcdboot` against the existing Windows and ESP volumes.
4. **Rollback last:** select the previous verified PXE release if a current installer or driver bundle is suspect. A PXE rollback does not modify an already installed workstation.

Answer before continuing: Which layer failed: firmware entry, selector file, OS loader, filesystem, or operating system? Which read-only observation proves that classification? Can the repair be performed without formatting or moving a partition? Where is the recovery evidence saved?

ESCALATION BOUNDARY

If neither filesystem is readable, the disk reports new media errors, or the observed partition boundaries differ from the signed plan, stop. Image the disk or replace the disposable pilot; do not use filesystem repair, partition reconstruction, or installation retry as an exploratory diagnostic.

Stage 7 — Join and test Windows

Join Windows 11 Pro to `⟨identity-domain⟩`, restart, and log in online once with an ordinary domain test account. Test the daily administrator separately; do not use the domain-wide administrator for ordinary desktop work.

whoami	Returns the short domain and the intended user.
whoami /groups	The ordinary test user lacks local Administrators; the approved workstation administrator has it.
klist	Kerberos tickets name the permanent realm.
Resolve-DnsName -Type SRV _ldap._tcp.⟨identity-domain⟩	The client discovers the DC through AD DNS rather than a hosts-file entry.

Disconnect the network and restart. The previously tested user must be able to open the cached local profile. Reconnect, disable the account in AD, and prove connected login is denied. Record that disconnected cached login remains a known phase-1 limitation.

Stage 8 — Join and test Arch

Configure SSSD against the same domain. Create local homes on first login; never make the base home path depend on network storage. Grant sudo only through an explicit AD workstation-administrator group.

id <test-user>	Returns a stable domain UID, primary group and expected memberships.
kinit <test-user> && klist	The realm ticket succeeds while online; destroy the ticket after recording the result.
getent passwd <test-user>	The NSS record and home path are present without editing local /etc/passwd.
sudo -l -U <test-user>	The ordinary test user has no sudo rule.

Repeat online login, offline cached login, connected account disable, DNS failure and time-skew tests. A friendly desktop symptom is not evidence: record command output and the exact expected boundary.

% Integration point: place after Stage 8's local-rescue prerequisite and % before optional storage. This file deliberately contains no document wrapper.

Join and prove Windows 11

Start on a wired provisioning connection. Log in as the tested local rescue administrator, set the permanent workstation name, and point the interface at AD DNS. Do not join while the client is using public, gateway, or fallback DNS.

```
Rename-Computer -NewName "<workstation-name>" -Restart
Get-DnsClientServerAddress -AddressFamily IPv4
Resolve-DnsName -Type SRV "_ldap._tcp.dc._msdcs.<identity-domain>"
w32tm /query /source
Test-NetConnection <dc-fqdn> -Port 445
```

Expect: The computer name is exact; every active domain-member interface lists only approved AD DNS servers; the SRV answer names a DC; time has a source; TCP 445 succeeds.

If not: If ordinary names resolve but the SRV query fails, correct DHCP or interface DNS. If the clock differs from the DC by five minutes or more, correct time before touching the domain. Do not work around either fault with a hosts-file entry.

Join from *Settings > System > About > Domain or workgroup > Change*, select *Domain*, enter `⟨identity-domain⟩`, and supply the temporary delegated join credential. A domain administrator is not required for routine joins. Restart when Windows asks.

```
Get-CimInstance Win32_ComputerSystem |
Select-Object Name,PartOfDomain,Domain
nltest /dsgetdc:<identity-domain>
Test-ComputerSecureChannel -Verbose
```

Expect: PartOfDomain is true, Domain is exact, nltest discovers a DC through DNS, and the secure channel reports True. Record the DC name and test time.

At the sign-in screen choose *Other user*. Sign in once as the standard test identity using DOMAIN\user or the full user principal name. Open a non-elevated PowerShell session:

```
whoami
whoami /groups
echo $env:USERPROFILE
net localgroup Administrators
```

The reported identity is the standard domain test user	Its profile is local, its SID is present, and it is absent from the local Administrators group.
Sign in as the approved daily administrator	A deliberate elevation prompt succeeds, but the reserved domain-administrator identity has not been used for daily work.
Disconnect every network interface and restart	The previously used domain identity signs in from its cached credential; network resources fail without delaying or preventing the local desktop.
Reconnect, disable the standard test identity, then attempt a fresh online sign-in	Connected authentication is denied. Record separately that an already cached offline credential cannot be revoked in phase 1.

Answer before continuing: Did the join use a delegated credential? Can local rescue still sign in with the DC powered off? Is the standard identity non-administrative? Did the test measure DNS answer, clock source, secure-channel state, and profile path rather than infer success from a desktop appearing?

Join and prove Arch Linux

Finish a full system update before the join; Arch does not support partial upgrades. Install the declared identity client as one transaction:

```
sudo pacman -Syu -needed sssd samba krb5 bind
resolvectl status
timedatectl show -p NTPSynchronized -value
host -t SRV _ldap._tcp.dc._msdcs.<identity-domain>
```

Expect: AD DNS is active, time reports yes, the LDAP SRV answer names a DC, and the packages came from enabled official Arch repositories.

If not: Stop if discovery fails, time is not synchronized, or Network Manager can replace AD DNS with a public resolver. A successful ping is not an identity-plane test.

Create a one-use delegated join identity or obtain a time-limited credential from the encrypted secret channel. Generate /etc/samba/smb.conf from private values with security = ads, the exact workgroup = <netbios-name> and realm = <kerberos-realm>. Set the Kerberos method to secrets and keytab. Generate root-owned mode-0600 /etc/sssd/sssd.conf with:

```
[sssd]
services = nss, pam
config_file_version = 2
domains = <identity-domain>
[domain/<identity-domain>]
id_provider = ad
auth_provider = ad
access_provider = ad
ad_domain = <identity-domain>
krb5_realm = <kerberos-realm>
cache_credentials = true
use_fully_qualified_names = true
fallback_homedir = /home/%u
default_shell = /bin/bash
ldap_id_mapping = true
```

Review both rendered files before joining. Join interactively; never place the password in a command, shell history, answer file, or repository.

```
sudo net ads join -U '<delegated-join-user>'
sudo net ads testjoin
sudo klist -k /etc/krb5.keytab
sudo test "$(stat -c %a /etc/sssd/sssd.conf)" = 600
sudo systemctl enable -now sssd
```

Expect: `net ads testjoin` reports success, the machine keytab contains the expected host principals, `/etc/sssd/sssd.conf` is root-owned mode 0600, and `sssd.service` is active. Revoke or expire the join credential now.

Keep the naming rule explicit. Either retain fully qualified login names (`user@domain`) or set a reviewed, site-wide short-name rule; never allow different domains or local accounts to collapse onto the same visible name. Confirm that `sss` appears on the `passwd`, `group`, and `shadow` lines in `/etc/nsswitch.conf`. Then verify lookups:

```
getent passwd '<standard-user>@<identity-domain>'
id '<standard-user>@<identity-domain>'
getent group '<approved-admin-group>@<identity-domain>'
sssctl user-checks '<standard-user>@<identity-domain>' -s login
```

Expect: The user resolves once, has a stable numeric UID and GID, is absent from the approved administrator group, and SSSD authorizes the login service. Record the numeric IDs; optional SMB files must present compatible ownership.

Local home creation is a PAM action, not a network-home mount. Before editing, save the package-owned file's digest and a private evidence copy. Add this line to the session stack used by the workstation login path, normally `/etc/pam.d/system-login`, after the other required session setup:

```
session optional pam_mkhomedir.so skel=/etc/skel umask=0077
```

Review the complete PAM stack before rebooting. Do not add `pam_mkhomedir` to authentication or account sections, and do not replace package-managed includes. Test first from a second TTY while the local rescue session remains open.

```
getent passwd '<standard-user>@<identity-domain>'
findmnt -target '/home/<resolved-home>'
stat -c '%U %G %a %n' '/home/<resolved-home>'
sudo -l -U '<standard-user>@<identity-domain>'
```

`First online login`	Creates exactly the home path returned by `getent`; it is a local filesystem path, owned by the resolved UID/GID, and begins with restrictive permissions.
`Standard-user authorization`	The user receives no workstation administration through `sudo`.
`Approved administrator authorization`	Only the reviewed domain group receives the intended narrow `sudo` policy; a real elevation is tested and logged.

`Offline restart after one successful online login`	The cached identity signs in, the same numeric UID and GID are returned, and the existing local home opens even when DNS, LDAP, Kerberos, and SMB are unavailable.
`Local rescue with the DC unavailable`	Login and `sudo` succeed without SSSD. If they do not, restore the PAM/NSS configuration from the still-open rescue session.

UID, TIME, AND STORAGE PROOF

Record UID, primary GID, supplementary groups, client time, DC time, and the SMB file owner observed from both operating systems. A name that merely looks correct is not proof. NFS remains disabled until numeric-ID mapping is designed and tested; phase 1 uses SMB and a local home.

Answer before continuing: What exact login spelling is canonical? Which AD group alone grants local administration? Does the home exist and open with storage powered off? Do UID and GID remain unchanged across two cold boots? Is clock offset below the Kerberos limit? Does disabling an account deny a new connected login while the documented offline-cache limitation remains understood?

Stage 9 — Add optional SMB storage

Integrate the NAS with AD before touching a workstation. Give each person a private share, or a folder whose ACL grants that AD person and the storage administrators access. Do not store a second copy of the person's password. The workstation presents the current Kerberos identity to SMB.

THE LOCAL PROFILE IS AUTHORITATIVE

Windows must reach its desktop and Arch must reach its local home without the NAS. Network storage is a convenience mounted after login. Never redirect `Desktop`, `Documents`, `%USERPROFILE%`, or `/home` during phase 1.

Prove the server before configuring a client

1. From AD DNS, resolve `⟨storage-host⟩`. Record the returned address and resolver. *This catches split-DNS and stale-record errors.*
2. With an ordinary test identity, list the intended share. Confirm that a different ordinary identity is denied. *This proves the server ACL instead of relying on a drive letter or desktop icon.*
3. Create a uniquely named test file, read it back, remove it, and record the server-side owner. *This detects guest fallback and identity-mapping errors.*
4. Compare client, domain-controller, and NAS time. Keep observed skew below the site's Kerberos limit. *SMB may look like an ACL failure when the ticket is actually too old or the clocks disagree.*

If not: If the owner is guest, an unexpected numeric identity, or another household user, stop. Correct the NAS domain join and ACL model before automating any workstation mount. Do not compensate with a shared credential or a permissive ACL.

Windows: map after desktop login

1. Log in online once as the ordinary domain test user. Run `klist`; obtain a ticket before continuing.
2. In File Explorer, map the approved UNC path to the documented drive letter. Select reconnect, but do not select alternate credentials and do not save a password in Credential Manager.
3. Put the mapping action in a per-user logon task with an eight-second timeout and no "wait for network" prerequisite. The task may report failure; it may not hold the desktop open. Record the task name and exported, secret-free definition.
4. Sign out and back in with the NAS reachable. Create a file, disconnect the mapping, reconnect it, and read the same bytes. Record the path, SHA-256 digest, elapsed login time, and effective AD owner.

`Get-SmbMapping`	The approved drive points to the approved UNC path, uses the signed-in domain identity, and reports no guest mapping.

`cmd /c "net use"`	No obsolete drive letter, alternate NAS name, or unexpected cached connection is present.

Arch: mount in the user session

Install the repository-declared SMB and desktop virtual-filesystem packages. Use the desktop's user-session SMB mount, such as GVfs, so no network filesystem participates in boot or local-home creation.

1. Log in online as the ordinary domain test user. Run `klist`; fix SSSD or Kerberos first if no current ticket exists.
2. Open `smb://⟨storage-host⟩/⟨user-share⟩` from the desktop file manager. Choose the current domain identity and decline permanent password storage.
3. If automatic connection is wanted, create a user-session service that runs the mount after the graphical session starts, has an eight-second timeout, and is not required by the session target. Never add the SMB path to `/etc/fstab`, the boot target, or the local-home unit.
4. Create, hash, unmount, remount, and read the same test file. Record the Kerberos principal and effective server-side owner.

`gio mount -l`	The approved SMB location appears only when reachable; the local home remains an ordinary local filesystem.
`systemctl -user -failed`	A failed optional mount may be visible, but no login, home, or graphical-session unit depends on it.

Run the three-state failure drill on both systems

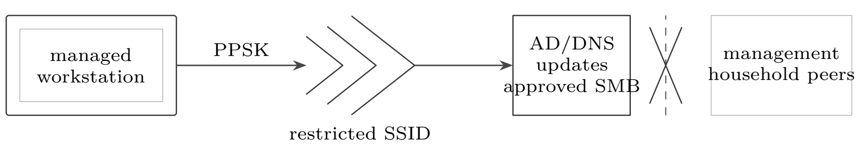
State	Action and measurement	Pass boundary
Available	Login, create and hash a file; unmount and remount	Same bytes, same AD owner, no password prompt
Unreachable	Block or disconnect only the NAS; cold-login with a stopwatch	Local desktop/home opens within the declared limit; bounded warning only
Denied	Keep the host reachable but remove the test user's share access	Explicit access-denied result; local login works; no guest or alternate user

Answer before continuing: Did each test begin from a cold sign-in rather than an already connected session? Was the NAS itself unreachable in the second test, but reachable in the third? Did either client silently reuse an old connection or credential?

If not: If login waits on storage, remove the mapping from every boot, profile, shell-startup and required-session dependency, then repeat all three states. If access denied becomes access granted after a prompt, remove the saved credential and inspect the server ACL.

Stage 10 — Add restricted managed Wi-Fi

Create a dedicated managed-workstation SSID and VLAN. Prefer one PPSK per workstation so one lost laptop can be removed without changing every machine. Retrieve the selected key from the private secret store only while minting the machine. Phase-1 disks are unencrypted; assume physical access can recover any stored wireless credential.



A stored credential reaches only the services a managed workstation needs.

Prove UniFi policy before storing a key

Using a temporary test client, verify association, DHCP lease, gateway, AD DNS, time, domain ports, approved SMB, and both operating systems' update destinations. Then verify denials to gateway administration, Controller administration, infrastructure management, household-client, peer-client, and IoT destinations. Save a small source/destination/port/result table.

If not: If an allow fails, inspect association, DHCP, DNS, time, then the single destination and port. If a deny fails, quarantine the SSID. Do not store its credential on a workstation until both directions match policy.

Windows: install one machine profile

1. Create a WLAN profile for only the managed SSID and its assigned PPSK. Import it for all users from the local console. Do not add household, management, or provisioning SSIDs.
2. Delete the plaintext import file immediately after the profile appears. Check Git status, the build log, answer files and shell history for the SSID key; store only the key's secret-manager reference in evidence.
3. Disconnect Ethernet, restart, and log in with a previously cached ordinary account. Confirm automatic association and the expected managed-VLAN lease.
4. Forget or disable the profile, restart, and prove that Windows does not fall back to another known private network. Restore only the approved profile.

`netsh wlan show profiles`	Exactly the approved preload set appears; captured output contains no key material.
`Get-NetConnectionProfile`	The active wireless interface has the expected network category and no second active route into a private zone.

Arch: install one root-owned NetworkManager profile

1. Create one system connection for the managed SSID with NetworkManager, binding it to the workstation PPSK and enabling autoconnect. Avoid typing the key in a command that will be retained in shell history.
2. Confirm its file under `/etc/NetworkManager/system-connections/` is owned by root, mode 600, and excluded from logs, artifacts, backups without secret encryption, and Git.
3. Disconnect Ethernet, restart NetworkManager or reboot, and log in with a previously cached ordinary account. Confirm association, expected lease, AD DNS, time, and the default route.
4. Set the connection down and prove that no household or management SSID autoconnects. Bring back only the approved profile.

`nmcli -f NAME,TYPE,AUTOCONNECT connection show`	Only approved wireless profiles autoconnect; do not include secret fields in evidence.
`stat -c '%U %G %a %n' /etc/NetworkManager/system-connections/*`	Every stored system profile is root-owned and mode 600.

Wireless-only acceptance

Repeat AD DNS SRV lookup, clock check, ordinary online login, administrator authorization, OS update check and optional-SMB three-state drill with Ethernet physically disconnected. Measure association-to-lease and login time. Scan each forbidden zone from only its documented small CIDR; do not substitute a wide private-range scan.

Answer before continuing: Does revoking this workstation's PPSK leave every other workstation connected? Does the revoked machine fail to associate after its cached radio state is cleared? Can it reach all required update endpoints but no management page or peer workstation?

If not: If wireless onboarding fails, continue over the deployment cable. Never inject the household Wi-Fi key or broaden the VLAN to make the test pass. If a required vendor update cannot be bounded yet, record that exception and keep the workstation a pilot.

Stage 11 — Publish and migrate

Publishing a PXE release uploads to a new immutable version directory, verifies the remote bytes with a checksum comparison, and atomically moves `current`. Rollback selects a previous verified release; it never edits that release.

```
make homelab-pxe-publish RELEASE=<local-release> \
DESTINATION=<host:/absolute/root>
make homelab-pxe-rollback TARGET=<target> VERSION=<previous-version> \
DESTINATION=<host:/absolute/root>
```

The permanent physical Controller later joins as `permanent_dc_fqdn`. Validate AD/DNS replication. Move the boot alias and UniFi boot settings, then demote `bootstrap-dc`. Workstations are not rebuilt because their permanent domain, realm and discovery mechanism do not change.

Future VM candidate	Samba AD/DNS can move behind the same names after an additional DC replicates and roles transfer.
Future VM candidate	A deployment credential broker can issue short-lived joins without placing reusable credentials in artifacts.
Future VM candidate	Windows deployment helpers, certificate services, audit collection and management services can be isolated later.
No rebuild trigger	A service move is acceptable only when an existing Windows and Arch client passes discovery and login unchanged.

Final acceptance worksheet

Complete this only after the procedure. Worksheets belong at the back so they support the instruction rather than interrupt its flow.

Claim	Pass / fail	Evidence location or measured value
One DHCP authority observed		
AD LDAP and Kerberos SRV records resolve		
Controller and client clocks synchronized		
PXE release version and manifest verified		
Disk serial matched before first write		
Windows/Arch allocation matches plan		
Windows is default after five seconds		
Ordinary user logs in on Windows		
Ordinary user logs in on Arch		
Ordinary user lacks admin and sudo		
Workstation administrator has intended rights		
Both systems permit tested offline login		
Connected disabled account is denied		
SMB unavailable does not block login		
No plaintext secret appears in Git or logs		

Workstation hostname: ______________________ Disk serial: ______________________

PXE release: ______________________ Operator/date: ______________________

RELEASE DECISION

Every row passes with evidence, or the workstation remains a disposable pilot. Record exceptions explicitly; familiarity, a successful desktop login, and "it worked once" are not acceptance evidence.

Exception and retest record

Record each failed gate and its evidence; strike unused rows.

Gate	Failure and measurement	Correction and retest evidence	Result

Evidence packet location/checksum: ______________________

Reviewer/date: ______________________ **Disposition: release / pilot only / rebuild**